

# Negación y Cortes

---

Robin Omar Buezo Díaz



Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

# Indice

## Contenido

|                                                 |   |
|-------------------------------------------------|---|
| Indice .....                                    | 1 |
| Competencia(s) .....                            | 2 |
| Implementación de negación en inferencias ..... | 2 |
| Objetivo.....                                   | 2 |
| Introducción.....                               | 2 |
| Contenido .....                                 | 2 |
| Pongamos en Práctica la Teoría .....            | 2 |
| Caso de Estudio 1.....                          | 2 |
| Caso de Estudio 2.....                          | 3 |
| Conclusiones .....                              | 3 |
| Control de flujo mediante cortes.....           | 3 |
| Objetivo.....                                   | 3 |
| Introducción.....                               | 3 |
| Contenido .....                                 | 3 |
| Pongamos en Práctica la Teoría .....            | 4 |
| Caso de Estudio 1.....                          | 4 |
| Caso de Estudio 2.....                          | 4 |
| Conclusión.....                                 | 4 |
| Conclusión General .....                        | 4 |
| Referencias .....                               | 4 |

## Competencia(s)

Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación y cortes en Tau Prolog para modelar bases de conocimiento y resolver problemas de inferencia lógica.

## Implementación de negación en inferencias

### Objetivo

Aprender a utilizar la negación lógica en reglas Prolog para representar condiciones que excluyen ciertos resultados.

### Introducción

En la lógica clásica, la negación representa la ausencia o falsedad de una afirmación. En Prolog, se trabaja bajo el principio de “negación por falla”, donde algo se considera falso si no puede probarse como verdadero. Este principio tiene implicaciones importantes cuando se diseña una base de conocimiento.

### Contenido

La negación en Prolog se expresa con el operador `\+`. Por ejemplo, para decir que un estudiante no aprobó:

```
no_aprueba(X) :- estudiante(X), \+ aprobado(X).
```

Esto significa que si no hay una prueba de que X aprobó, se asume que no lo hizo.

Es importante notar que esta negación no significa “lógicamente falso” en todos los casos. Solo falla si no puede demostrarse.

## Pongamos en Práctica la Teoría

### Caso de Estudio 1

En un sistema bancario se quiere identificar a los clientes sin deudas:

```
cliente_activo(X) :- cliente(X), \+ deuda(X, _).
```

Esta regla verifica si un cliente no tiene ningún registro de deuda.

## Caso de Estudio 2

En una red social educativa se desea evitar que un usuario acceda si ya está bloqueado:

```
puede_entrar(X) :- usuario(X), \+ bloqueado(X).
```

Esta regla niega la posibilidad de acceso si el usuario tiene bloqueo registrado.

## Conclusiones

La negación permite representar restricciones o condiciones excluyentes dentro de la lógica declarativa. Es útil para modelar situaciones en las que algo solo ocurre si otra cosa no lo ha hecho.

## Control de flujo mediante cortes

### Objetivo

Aplicar el operador de corte en reglas Prolog para evitar backtracking innecesario y controlar el flujo de ejecución lógica.

### Introducción

Prolog utiliza backtracking para explorar alternativas. En algunos casos, esto genera respuestas repetidas o evaluaciones innecesarias. Para evitarlo, se puede usar el operador de corte `!`, que actúa como una barrera que impide retroceder en el proceso de evaluación una vez pasada.

### Contenido

El corte se coloca dentro de una regla para indicar que si las condiciones antes del corte se cumplen, las siguientes deben ejecutarse sin volver atrás.

Ejemplo:

```
clasifica(X, bajo) :- nota(X, N), N < 61, !.
```

```
clasifica(X, medio) :- nota(X, N), N < 81, !.
```

```
clasifica(X, alto).
```

Aquí, si un valor entra en la primera o segunda categoría, Prolog no buscará otras respuestas.

El corte también se utiliza para optimizar procesos que no requieren evaluar más de una solución.

## Pongamos en Práctica la Teoría

### Caso de Estudio 1

Un sistema de control de acceso solo verifica una vez por sesión. Se aplica:

```
permitir(X) :- sesion_valida(X), !, acceso(X).
```

El corte evita que se vuelva a validar la sesión si ya fue aceptada.

### Caso de Estudio 2

En una calculadora lógica se puede usar el corte para evitar que se busquen más formas de calcular cuando ya se encontró una.

```
resultado(X, Y, suma, R) :- R is X + Y, !.
```

## Conclusión

El uso de cortes permite tener mayor control sobre cómo se comportan las reglas. Evita respuestas múltiples innecesarias y mejora la eficiencia del sistema, pero su uso debe ser cuidadoso para no restringir demasiado la lógica.

## Conclusión General

La negación y el corte son dos herramientas fundamentales en la programación lógica avanzada. Permiten excluir respuestas no deseadas, evitar caminos lógicos redundantes y controlar el flujo de inferencia para hacer más eficiente la base de conocimiento.

## Referencias

Bratko, I. (2012). *Prolog Programming for Artificial Intelligence*. Pearson  
Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog*. Springer  
Nilsson, N. J. (1998). *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann

